Prerequisites and objectives

- Prerequisites are BSTs, rotations, recursion, probability-weighted costs, dynamic programming, and amortized analysis.
- Distinguish optimal BST's static weighted objective, AVL and red-black worst-case height guarantees, and splay sequence guarantees.
- Check rotations using inorder order and structural invariants instead of memorizing shapes alone.

Which cost is being optimized?

- Minimizing height reduces worst-case search depth but need not minimize average search cost.
- If key k_i is queried with known probability p_i, expected successful comparisons are $\sum_i p_i(\text{depth}(k_i) + 1)$.
- Accounting for unsuccessful queries requires n+1 gap probabilities $q_0, \ldots, q_n$ and a stated dummy-leaf cost convention.
- Optimal BSTs suit relatively stable distributions and static indexes. Frequent updates require separately accounting for reoptimization.

Successful-search optimal BST recurrence

- Let C[i, j] be the minimum weighted comparison cost for ordered keys in closed interval [i, j]. Empty intervals cost 0.
- Define $W[i, j] = \sum_{k=i}^{j} p_k$. Choosing root r adds one comparison to every key in the interval relative to its child-subproblem cost.
- Thus `C[i,j]=W[i,j]+min_{r=i..j}(C[i,r-1]+C[r+1,j])`.
- Fill increasing interval lengths. Prefix sums give O(1) W queries. Time is $O(n^3)$, space $O(n^2)$. Store optimal roots to reconstruct the tree.

Worked weighted examples

- For probabilities p=[0.1,0.2,0.7], root key 2 gives cost 1+0.1+0.7=1.8.
- Root key 3 with key 2 at its left and key 1 below that gives 0.7 • 1+0.2 • 2+0.1 • 3=1.4.
- The minimum-height choice is not the minimum-expected-cost choice. Placing a frequent key near the root can offset deeper rare keys.
- Greedily taking the highest-probability key is still not always optimal. For p=[0.4,0.35,0.25], the best tree rooted at key 1 costs 1.85, while root key 2 gives 1.65.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A standard unsuccessful-search convention

- Set `e[i,i-1]=q_{i-1}`, charging one dummy comparison in an empty subtree. Define $w[i,j]=\sum_{k=i}^{j} p_k+\sum_{k=i-1}^{j} q_k$.
- The recurrence is `e[i,j]=w[i,j]+min_r(e[i,r-1]+e[r+1,j])`. Under this convention the empty cost cannot be 0.
- This computes $\sum_i p_i(\mathrm{depth}(\mathrm{key}_i)+1)+\sum_j q_j(\mathrm{depth}(\mathrm{dummy}_j)+1)$. Counting only real-key comparisons subtracts one per failed query, a constant total $\sum_j q_j$, so the optimal tree is unchanged.
- Knuth's O(n^2) optimization requires the applicable nonnegative weights and monotone-root property. An arbitrary interval DP does not inherit it merely by name. Its full proof is outside this edition.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete optimal-BST interval table and failed-query example

```
for i=1..n+1: C[i,i-1]=0
for length=1..n:
    for i=1..n-length+1:
        j=i+length-1; C[i,j]=infinity
        for r=i..j:
            candidate=W(i,j)+C[i,r-1]+C[r+1,j]
            if candidate<C[i,j]: C[i,j]=candidate; root[i,j]=r
build(i,j):
    if i>j: return null
    r=root[i,j]; return Node(k[r],build(i,r-1),build(r+1,j))
```

For successful probabilities [.1,.2,.7], singleton costs are .1,.2,.7. C12 candidates are .5,.4; C23 candidates are 1.6,1.1; C13 candidates are 2.1,1.8,1.4. Reconstruct root 3 with left child 2 and left grandchild 1. Direct weighted comparisons give .1×3+.2×2+.7×1=1.4.

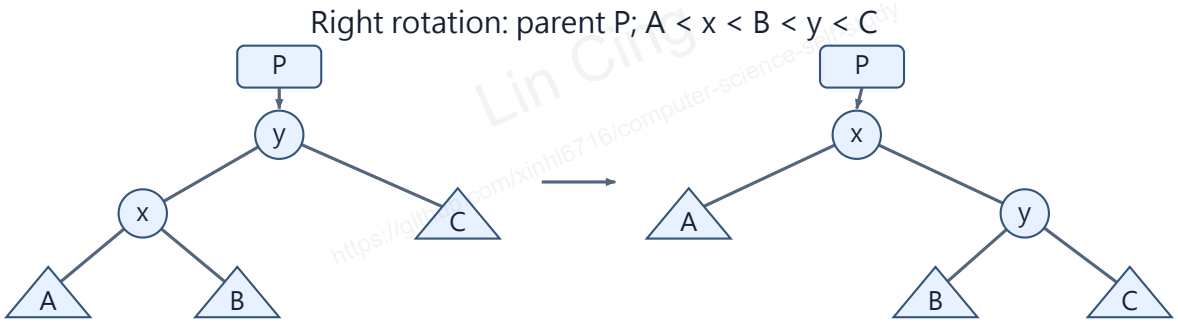
For one key with p1=.6, q0=.1, q1=.3, dummy-counting initialization e10=.1, e21=.3, w11=1 yields e11=1.4. Counting only real comparisons gives 1 because every query compares the sole real key once. The difference is q0+q1=.4. Do not initialize empty costs to 0 and still claim the dummy-counting result.

Complete optimal-BST interval table and failed-query example (continued)

For a fixed root r, ordering uniquely partitions key ranges. Replacing a nonoptimal child tree by an optimal tree on the same keys lowers total cost without changing root contribution W, contradicting optimality. Induction on interval length proves the DP without a highest-frequency greedy assumption.

Rotations preserve inorder order

- Before a right rotation, y is root and x its left child, with order `A<x<B<y<C`. Afterward x is root and y its right child. Inorder order is unchanged.
- Preserve B, set y.left=B and x.right=y, then reconnect the original parent. If parents are stored, update B and both rotated nodes' parent fields.
- Update height, subtree size, or other augmentation first at lowered y, then at raised x.
- Rotation preserves BST order, not every extra invariant. Colors and AVL heights require their own repair.

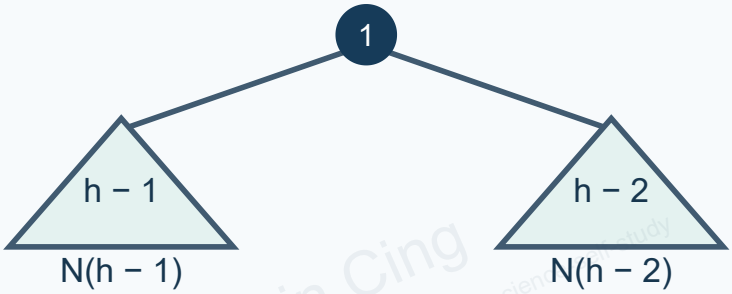


AVL local height constraint

- Set h(null)=−1 and h(leaf)=0. Balance factor `bf(x)=h(left)-h(right)` must be −1,0, or 1.
- Minimum nodes at height h satisfy `N(h)=1+N(h-1)+N(h-2)`, for h≥1, with N(−1)=0, N(0)=1. The taller child has height h−1 and the other at least h−2.
- Fibonacci growth gives h=O(log n). Search, insertion, and deletion have O(log n) worst-case path lengths.
- The constraint holds at every node, not only the root.

The AVL height bound: why the recurrence is exact

Set the empty-tree height to −1 and leaf height to 0. Let N(h) be the fewest nodes in an AVL tree of height exactly h.



At least one child has height h−1, and balance requires the other to have height at least h−2. Choosing minimum trees of both heights attains the lower bound, so $N(h)=1+N(h-1)+N(h-2)$. With $N(-1)=0$, $N(0)=1$, $F_0=0$ and $F_1=1$, induction gives $N(h)=F_{h+3}-1$.

The height of a 200-node AVL tree

Edge height h	7	8	9	10
Minimum N(h)	54	88	143	232

Height 10 requires at least 232 nodes, so 200 nodes have height at most 9. The bound is attained with two height-8 children of sizes 88 and 111 plus the root. Start with a minimum 88-node height-8 AVL tree and insert 23 distinct keys. The height cannot become 9 while the total is less than N(9)=143, so a height-8 tree of size 111 exists.

If leaf height is defined as 1 instead, the same tree has 10 levels. Always state whether height counts edges or node levels.

Four AVL insertion shapes

- LL means a left-heavy x with insertion in its left child's left side. Right-rotate x. RR is the symmetric left rotation.
- LR first left-rotates the left child, then right-rotates x. RL is symmetric.
- Inserting 30,10,20 produces LR. Raise 20 above 10, then above 30, giving root 20 with children 10,30.
- Repair at the first unbalanced insertion ancestor restores that subtree's preinsertion height. No further balancing rotations are required above it, but metadata must still be correct.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

AVL deletion may repair repeatedly

- First perform BST deletion. Update toward the root from the position where height actually decreased. Local repair need not restore predeletion height.
- If bf(x)=2 and bf(left)≥0, right-rotate. If the left child's factor is negative, use LR. Right-heavy cases are symmetric.
- The deletion case with child bf=0 cannot be handled by remembering only insertion direction.
- At most O(log n) local updates along the root path give O(log n) worst-case total time. Stopping after one repair can leave a higher ancestor unbalanced.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

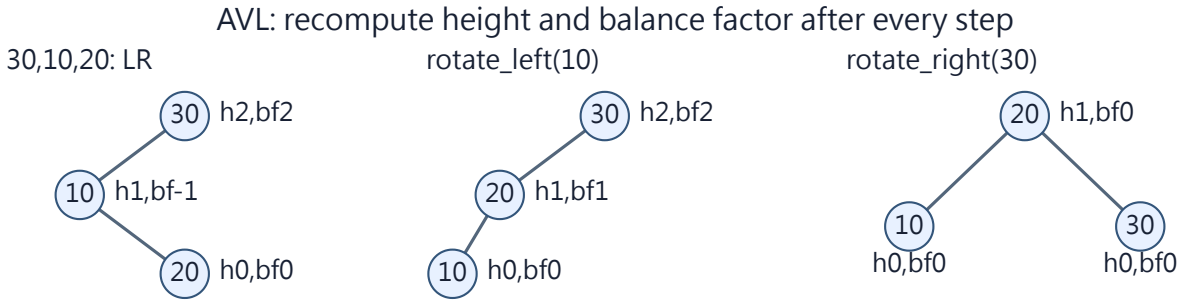
AVL recomputation and insertion/deletion repair

```
fix(x):
    if x==null: return null
    recompute_height(x)
    if bf(x)==2:
        if bf(x.left)<0: x.left=rotate_left(x.left)
        return rotate_right(x)
    if bf(x)==-2:
        if bf(x.right)>0: x.right=rotate_right(x.right)
        return rotate_left(x)
    return x
insert(x,k):
    if x==null: return leaf(k)
    if k<x.key: x.left=insert(x.left,k)
    else if k>x.key: x.right=insert(x.right,k)
```


AVL recomputation and insertion/deletion repair (continued)

```
    else: update/reject duplicate
    return fix(x)
erase(x,k):
    if x==null: return null
    if k<x.key: x.left=erase(x.left,k)
    else if k>x.key: x.right=erase(x.right,k)
    else:
        if x has at most one child:
            child=the nonnull child or null; release(x); return child
        s=minimum(x.right); x.contents=copy(s.contents)
        x.right=erase(x.right,s.key)
    return fix(x)
```

AVL recomputation and insertion/deletion repair (continued)



Each rotation recomputes the lowered node before the new root. For 30,10,20, initially 30 has (h2,bf2), 10 has (h1,bf−1), and 20 has (h0,bf0). Left rotation at 10 leaves 30 at (h2,bf2), with left child 20 at (h1,bf1). Right rotation at 30 yields root 20 at (h1,bf0) and two height-0 leaves.

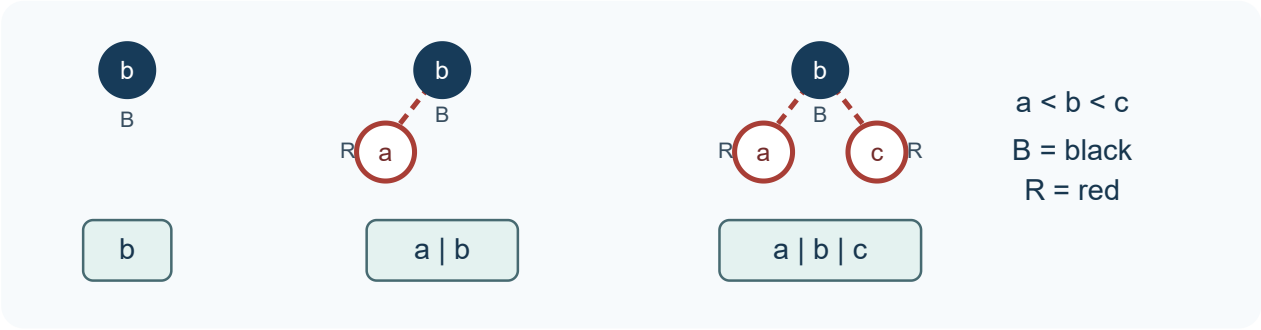
For root 4 with left subtree 2(1,3) and right leaf 5, deleting 5 gives bf(4)=2 and bf(2)=0. A single right rotation gives root 2, left child 1, right child 4, and 4.left=3. Local height stays 2 in this case. Other deletion repairs reduce height and must continue upward. Constant rotations and metadata work per logarithmic-depth frame give O(log n).

General red-black invariants

- Every node is red or black, the root is black, and all NIL leaves are black.
- Children of red nodes are black. Every path from a node to a descendant NIL contains the same number of black nodes.
- This chapter uses general node-colored red-black trees. It does not require all red links to lean left. Princeton's LLRB imposes a stronger representation convention.
- Include implicit NIL leaves in the black-height convention when checking a diagram.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Red-black trees and 2–3–4 trees

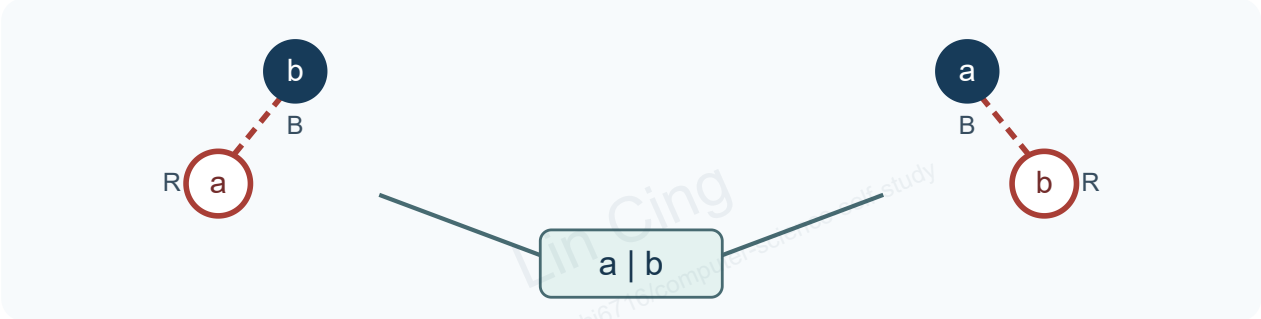


Merge each red node with its black parent. Zero red children give a 2-node; one gives a 3-node; two give a 4-node. Filled nodes are marked B, hollow nodes R, and links to red children are dashed, so the figure remains distinguishable in grayscale.

A red node cannot have a red child, so a merged node contains at most three keys. Each merged node on a path contributes one black node. Equal black heights therefore give equal depths for external leaves in the multiway tree. BST inorder order determines the interval of each child subtree.

A 2–3–4 tree can have multiple red-black encodings

A 3-node with a<b can be encoded as black b with red left child a, or black a with red right child b. Both contract to the same multiway node. General red-black trees permit either orientation. Left-leaning red-black trees add direction rules, and allowed intermediate states depend on the implementation.



An AVL LR or RL repair contains two primitive rotations. Some authors call the whole repair one double rotation; others count two rotations. State the unit before comparing counts. Every intermediate tree must retain BST order.

Red-black height bound

- Define bh(x) as black nodes strictly below x through and including NIL. A subtree with bh=b has at least $2^b - 1$ real nodes.
- For induction, each real child's black height is at least b−1. Its subtree therefore has at least $2^{b-1} - 1$ nodes. Adding both children and x yields at least $2^b - 1$. NIL gives the zero-node base.
- Red nodes cannot be consecutive, so root-to-NIL length is at most roughly twice the root black height. Combining $n \geq 2^{\mathrm{bh}(\mathrm{root})} - 1$ gives the standard height bound $h \leq 2\log_2(n + 1)$, with endpoint constants consistent with edge height.
- This is a worst-case guarantee with no random-key or uniform-query requirement.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Red-black insertion repair

- Color the new data leaf red, preserving black height. A black parent needs no repair. A red parent creates a red-red violation.
- If the uncle is red, color parent and uncle black, color grandparent red, and continue at the grandparent.
- With a black uncle, first rotate the parent if needed to turn a bent path into a straight one. Rotate the grandparent and exchange the appropriate colors. Mirror cases are symmetric.
- Finally color the root black. Inserting 10,5,1 right-rotates 10, yielding black root 5 with red children 1,10.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete ancestor loop for red-black insertion

```
insert_fix(z):
    while color(z.parent)==red:
        p=z.parent; g=p.parent
        if p==g.left:
            u=g.right
            if color(u)==red:
                p.black; u.black; g.red; z=g
            else:
                if z==p.right:
                    z=p; rotate_left(z); p=z.parent; g=p.parent
                p.black; g.red; rotate_right(g)
        else: apply exact left/right mirror
    root.black
```

New data leaves are red with black NIL children. Use a black parent sentinel for the root or explicitly guard null parents. Insert 10 as a black root, 5 as a red left child, then 1. A black uncle gives the straight case. Right-rotate 10 to obtain black root 5 with red children 1,10. Insert 7 under 10. Red parent 10 and uncle 1 become black, and grandparent 5 becomes red before the root becomes black again. Every root-to-NIL path has three blacks.

Complete ancestor loop for red-black insertion (continued)

A red uncle can propagate z repeatedly. In the bent black-uncle case, recompute p/g after the first rotation. Using stale parents can misattach the second rotation. Each loop moves upward or ends with constant rotations, so total time is O(log n). These branches implement general red-black invariants, not LLRB's stronger leaning rule.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Deletion and a black-height deficit

- Removing a red node does not reduce black height. If a removed black node has a red replacement child, color that child black.
- Otherwise track one missing black at replacement position x. This is an analysis state, not a required third stored color.
- Suppose x is a left child with sibling w. If w is red, rotate and recolor to obtain a black sibling before proceeding.
- If w is black with two black children, color w red and move the deficit to the parent. A previously red parent can become black and absorb the deficit.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Near and far nephews in deletion

- With x on the left, w's left child is the near nephew and its right child is the far nephew.
- If w is black, the far nephew black, and the near nephew red, right-rotate w and recolor, producing the far-red case.
- With a red far nephew, give w the parent's old color, color parent and far nephew black, and left-rotate the parent. This removes the deficit on x's side.
- All cases mirror when x is on the right. Each step does constant local work or moves upward, giving O(log n) worst-case repair.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rotation and the transplant entry to red-black deletion

```
right_rotate(y):
    x=y.left; require x!=NIL
    b=x.right; p=y.parent
    y.left=b
    if b!=NIL: b.parent=y
    x.parent=p
    if p==NIL: root=x
    else if p.left==y: p.left=x
    else: p.right=x
    x.right=y; y.parent=x
    pull(y); pull(x)
transplant(u,v):
    if u.parent==NIL: root=v
    else if u==u.parent.left: u.parent.left=v
    else: u.parent.right=v
    v.parent=u.parent       // including NIL: deletion needs its deficit parent
RB_erase(z):
```


Rotation and the transplant entry to red-black deletion (continued)

```
    y=z; removed_color=y.color
    if z.left==NIL:
        x=z.right; p=z.parent; transplant(z,x); pull_up(p)
    else if z.right==NIL:
        x=z.left; p=z.parent; transplant(z,x); pull_up(p)
    else:
        y=minimum(z.right); removed_color=y.color; x=y.right
        if y.parent==z: x.parent=y
        else:
            p_old=y.parent; transplant(y,x)
            pull_up(p_old)          // repair old path before y changes parent
            y.right=z.right; y.right.parent=y
        transplant(z,y); y.left=z.left; y.left.parent=y; y.color=z.color
        pull(y); pull_up(y.parent)  // repair y at its new position
    release(z)
    if removed_color==black: delete_fix(x)
```

Rotation and the transplant entry to red-black deletion (continued)

This page uses a shared black NIL sentinel rather than a data node. The root's parent is NIL, NIL is always black, and delete_fix is the four-case loop in "Complete control flow for red-black deletion repair". Left rotation is the exact left/right mirror. The complete augmentation here is subtree size: NIL.size=0 and pull(x)=1+size(left)+size(right); pull_up recomputes bottom-up until NIL; pure red-black metadata needs no automatic recoloring during rotation. Save middle subtree b and external parent p before modifying links.

Two-child deletion moves successor node y into z's position rather than copying a key and releasing y. The color removed from y's old position determines the black deficit, not the color y receives from z. Node x is y's sole right child or NIL. If y directly follows z, assign x.parent=y. Otherwise transplant y's right child into its old location, then attach z's right subtree to y. Attach the left subtree and original z color, and release z. All other element identities remain intact.

All empty children may share NIL, but deletion repair needs the parent of the current deficient x. Rotation does not overwrite NIL.parent for an arbitrary empty middle subtree b. Only an explicit transplant or direct-successor branch sets that parent. Deleting the sole black root gives root=NIL; delete_fix skips its loop because x==root, leaving a valid empty tree.

Rotation and the transplant entry to red-black deletion (continued)

The proof separates responsibilities: BST transplant preserves inorder and all undeleted elements; copying z's color preserves black counts at y's new location; only y's old position can have a deficit. The four-case loop preserves every other subtree while propagating or eliminating that single deficit. Search, successor lookup, and repair follow O(log n) paths; each transplant or rotation uses constant pointer work. The two additional pull_up paths each take O(log n) and cannot be omitted.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Subtree sizes, selection, and both repair paths

```
pull(x):
    require x!=NIL
    x.size=1+x.left.size+x.right.size
pull_up(x):
    while x!=NIL: pull(x); x=x.parent
select(k):
    require 1<=k<=root.size          // one-based rank
    x=root
    while x!=NIL:
        r=x.left.size+1
        if k==r: return x
        if k<r: x=x.left
        else: k-=r; x=x.right
    unreachable                     // sizes and rank invariant forbid this
```

Every data node satisfies size(x)=1+size(x.left)+size(x.right), with NIL.size=0. A new leaf starts at size 1. After BST insertion, pull_up from its parent before insert_fix. Each rotation pulls the descending node before the new local root.

Insert 4,2,6,1,3,5,7 legally, then erase red leaf 1. No color repair or rotation is needed, yet metadata must change:

Subtree sizes, selection, and both repair paths (continued)

Field	Before deletion	Transplant without pulls	Correctly updated
size(2)	3	3	2
size(4)	7	7	6
select(3)	Node 3	Reaches NIL	Node 4

With stale left size 3, root 4 claims rank 4 and sends rank 3 left. Node 2 then leads the search through node 3 to NIL. With correct left size 2, root 4 is exactly rank 3. Valid red-black colors and black heights do not establish valid augmentation.

For at most one child, transplant changes subtree sets only at z's old parent and above; pull_up from saved p. For a detached successor y, transplant its right child x into the old position, then immediately pull_up from p_old while that ancestor chain is intact. Move y to z's position, attach both children, pull(y), and pull_up from its new parent. A direct successor omits the old-position path but still needs the new-position repair. Overlapping paths add only a constant factor.

Subtree sizes, selection, and both repair paths (continued)

Bottom-up induction proves sizes: subtrees outside the changed paths are unchanged, and each pull sees already correct child sizes. Subsequent recoloring leaves sizes unchanged. A rotation changes child sets only at two local roots; pulling the lower before the upper fixes them. The local subtree's total element set is unchanged, so its external ancestors keep correct sizes. Selection maintains that k is a valid desired rank within the current subtree; exact left sizes justify each discarded interval, and finite height ensures termination.

Two repair paths, constant rotations, search, and successor lookup retain O(log n) worst-case update time; select is also O(log n). Height augmentation requires separate upward propagation when a rotation changes local height; the unchanged-element-count argument is specific to size.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete control flow for red-black deletion repair

```
while x!=root and color(x)==black:
    if x==x.parent.left:
        p=x.parent; w=p.right
        if color(w)==red:
            w.black; p.red; rotate_left(p); w=p.right
        if color(w.left)==black and color(w.right)==black:
            w.red; x=p
        else:
            if color(w.right)==black:
                w.left.black; w.red; rotate_right(w); w=p.right
            w.color=p.color; p.black; w.right.black
            rotate_left(p); x=root
    else: apply exact left/right mirror
x.black
```

Assume BST deletion/transplant is complete and x identifies the deficient position, possibly a NIL sentinel with its parent available. Skip repair if the physically removed color was red. Color assignments create no nodes. Rotations update external parents and root.

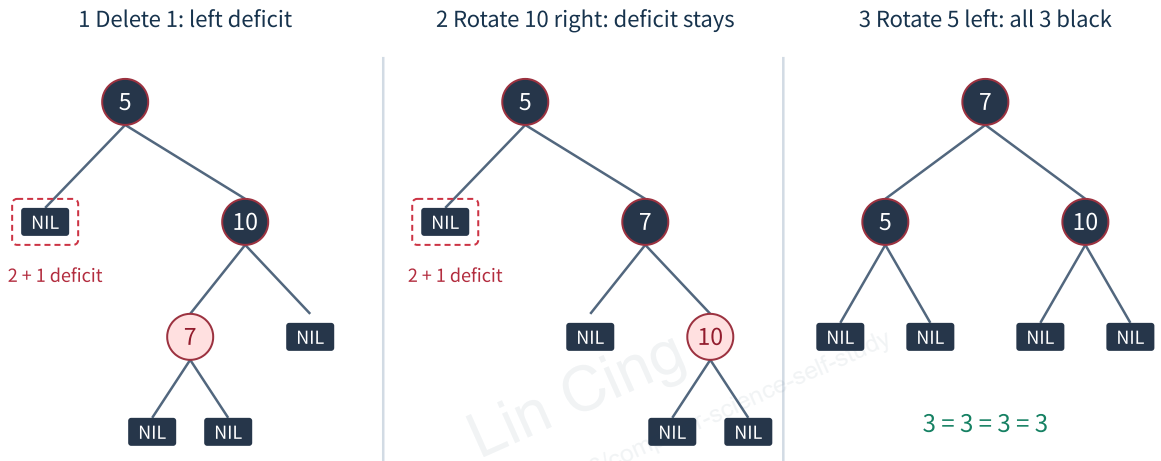
Complete control flow for red-black deletion repair (continued)

Use black root 5, black left child 1, black right child 10, and red node 7 as 10.left. Delete 1, leaving a deficient left NIL, black sibling 10, red near nephew 7, and black far NIL. Color 7 black and 10 red, then right-rotate 10. The new sibling 7 is black with red far nephew 10. Give 7 the old parent color, color 5 and 10 black, then left-rotate 5. The final black root 7 has black children 5,10. Every root-to-NIL path has three blacks, and inorder changes [1,5,7,10]→[5,7,10].

A red sibling conversion does not itself eliminate the deficit. Two black nephews can propagate it to the root. The cases cannot be replaced by unrelated one-level early returns.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Red-black deletion: deficit and two rotations



B: black / R: red; NIL is black. Inorder: 5,7,10.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Red-black deletion: deficit and two rotations (continued)

Start with the valid tree 5B(1B,10B(7R,NIL)) and delete 1. All NIL nodes are black; the dashed red outline denotes an analytical one-black deficit, not a third node color. In the first state the left route contains only 5B and NIL, two actual black nodes, while every right route contains three. Adding the deficit makes the counts temporarily equal. In state 2, recolor near nephew 7 black and 10 red, then rotate 10 right. The deficit remains at 5's left NIL. In state 3, 7 inherits 5's black color, 5 and far nephew 10 become black, and a left rotation at 5 makes all four root-to-NIL routes contain three black nodes. Only then is the deficit gone.

Every rotation preserves inorder [5,7,10] and updates the external root connection and parent pointers. These path counts include the local root and NIL, unlike the earlier bh(x) convention excluding x. Stopping after state 2 leaves the black deficit unresolved: converting a near red nephew into a far red nephew is a preparatory case, not completion.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splay zig, zig-zig, and zig-zag

- After accessing x, rotate it to the root. If only a parent remains, perform one zig.
- If x and its parent are both left children or both right children, zig-zig first rotates the parent over the grandparent, then x over its parent.
- With opposite directions, zig-zag first rotates x over its parent, then x over its grandparent.
- These preserve inorder order without a fixed balance factor or an O(log n) height guarantee after every operation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

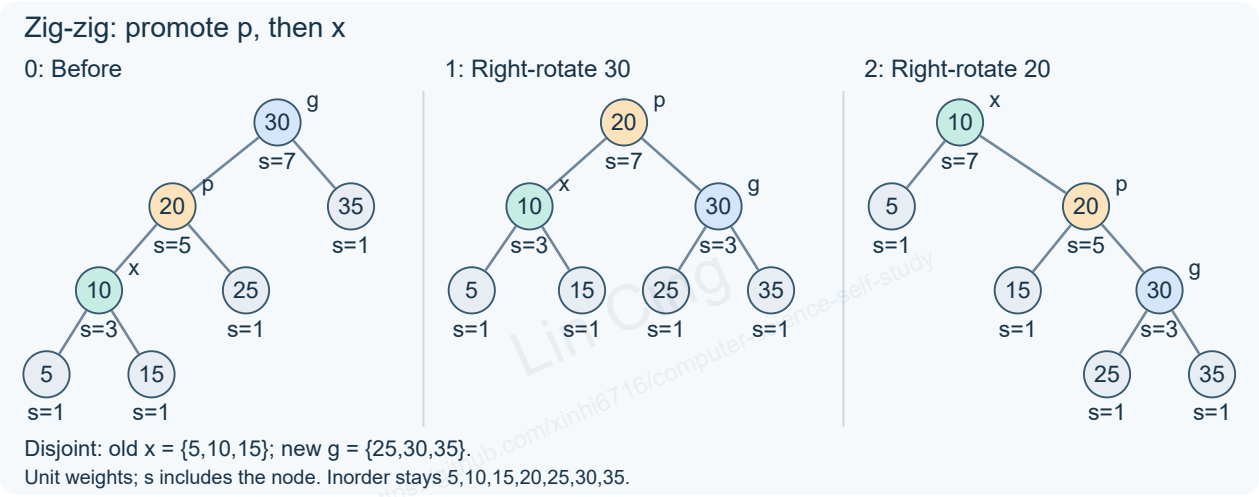
The splay access lemma

- Assign positive fixed node weights. Let s(x) be subtree weight, rank $r(x) = \log_2 s(x)$, and potential $\Phi = \sum_x r(x)$.
- The access lemma bounds a splay's amortized work by $O(1 + \log(s(\text{root})/s(x)))$.
- Unit weights give amortized O(log n) operations. An arbitrary initial tree requires an initial-potential term. A single operation is not worst-case logarithmic.
- In the right chain 1→2→3→4, searching for 4 visits four nodes. Repeated access after 4 reaches the root costs constant work, illustrating sequence adaptation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zig-zig: promote the parent first

Fix x=10, p=20, g=30. Promote orange p over g, then green x over p. The letters identify the same nodes throughout; do not rename them after a rotation.



The intermediate root is 20; only the final root is 10. Rotating x over p first and then over g produces a different tree. Moving the target to the root alone does not specify splay's rotation order.

A complete potential account for zig-zig

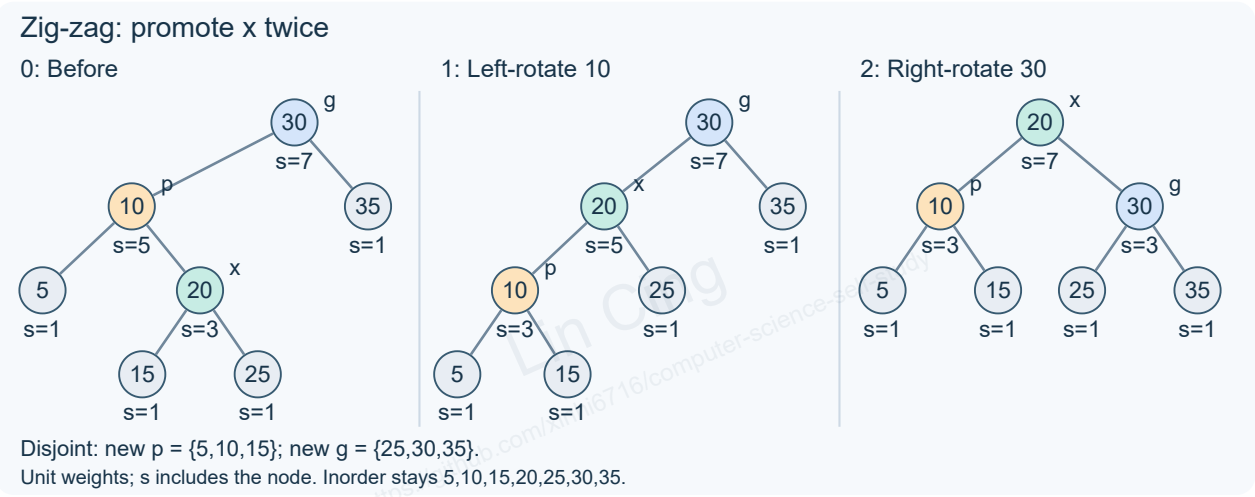
Use unit weights: s(v) counts the subtree including v, and $\Phi = \sum_v \log_2 s(v)$. The four leaf ranks stay zero; only three nodes need accounting.

Node	Initial s	After right-rotate 30	After right-rotate 20
x=10	3	3	7
p=20	5	7	5
g=30	7	3	3

Both endpoint potentials equal $\log_2(3 \cdot 5 \cdot 7) = \log_2 105$. Hence ΔΦ=0 and amortized rotation cost is 2. The lemma gives $3\log_2(7/3) \approx 3.667$, which exceeds 2. **Potential need not decrease on every operation; its changes account for the whole sequence.**

Zig-zag: promote the target twice

Fix x=20, p=10, g=30. Left-rotate 10, then right-rotate 30. Restore the four leaf attachments at each step.



Disjoint: new p = {5,10,15}; new g = {25,30,35}.
Unit weights; s includes the node. Inorder stays 5,10,15,20,25,30,35.

In the final tree the subtrees rooted at p and g are disjoint. These are the two sets needed for the concavity inequality on the next page.

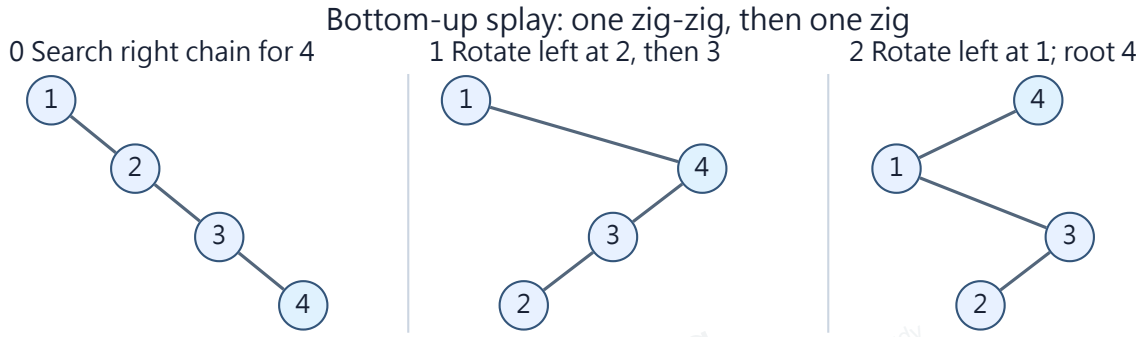
Connect the zig-zag numbers to concavity

Node	Initial s	After left-rotate 10	After right-rotate 30
x=20	3	5	7
p=10	5	3	3
g=30	7	7	3

$\Delta\Phi = \log_2(7 \cdot 3 \cdot 3) - \log_2(3 \cdot 5 \cdot 7) = \log_2(3/5) \approx -0.737$. Thus amortized cost is about 1.263, below $2\log_2(7/3) \approx 2.445$.

Here a=b=3 and a+b=6≤S=7. Since $(a-b)^2 \geq 0$, $ab \leq ((a+b)/2)^2 \leq (S/2)^2$. Taking base-2 logs yields $\log_2 a + \log_2 b \leq 2\log_2 S - 2$. Positive weights make the logs defined. **The example checks values; the general argument holds for arbitrary positive a and b.**

From a right-chain search to a complete splay



Search visits 1,2,3,4. Splay left-rotates 2, then 3, then 1. Finally 4.left=1, 1.right=3, 3.left=2; inorder remains [1,2,3,4]. A repeated search for 4 visits only the root and performs no rotations. Endpoint potentials both equal $\log_2 24$; the amortized guarantee does not require a decrease at every access.

Local accounting for the access lemma

- Use fixed positive weights, base-2 ranks, and one cost unit per rotation.
- Let p and g be x's parent and grandparent, and primes denote values after a double rotation. Since r′(x)=r(g) and all other node subtree weights are unchanged, amortized double-rotation cost is $2 + r'(p) + r'(g) - r(x) - r(p)$.
- For disjoint subtree weights a, b with a+b≤S, concavity gives $\log_2 a + \log_2 b \leq 2\log_2 S - 2$. The next two slides use this inequality to pay for both double-rotation cases.
- This proof counts rotations plus potential change. Constant pointer-work factors can be paid by scaling potential. Search depth equals rotation count up to endpoint work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zig-zig amortized inequality

- In zig-zig, the old x subtree and new g subtree are disjoint and both lie in the new x subtree. Thus $r(x) + r'(g) \leq 2r'(x) - 2$.
- Since r(p)≥r(x) and r′(p)≤r′(x), the double-rotation cost is at most $2 + r'(x) + r'(g) - 2r(x)$.
- Substituting the first inequality gives $\hat{c} \leq 3(r'(x) - r(x))$. These rank differences can be summed without assuming every rotation reduces global height.
- Identify the disjoint subtrees before applying concavity. Two immediate rotations alone do not prove constant amortized work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Zig-zag, zig, and telescoping

- After zig-zag, p and g subtrees are disjoint, so $r'(p) + r'(g) \leq 2r'(x) - 2$. With r(p)≥r(x), cost is at most $2(r'(x) - r(x))$, and therefore at most three times the difference.
- A final zig costs $1 + r'(p) - r(x) \leq 1 + r'(x) - r(x) \leq 1 + 3(r'(x) - r(x))$.
- The x-rank differences telescope. Amortized rotations for the whole splay are at most $3(\log_2 W - \log_2 s_{\text{before}}(x)) + 1$, where W is total tree weight.
- With unit weights, W=n and $s_{\text{before}}(x) \geq 1$, giving amortized O(log n). One Θ(n) operation is still allowed because actual totals include initial minus final potential.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Splay insertion, deletion, and constraints

- Insert a new BST leaf and splay it to the root.
- To delete, splay the target to the root and remove it, leaving L and R. If L is nonempty, splay its maximum to the root. Its right child is empty, so attach R there.
- An unsuccessful search may splay the last visited node, adapting to the query sequence.
- Reads modify structure. Concurrent read-only access and snapshots require additional design. Strict per-operation latency requirements favor worst-case balanced trees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete splay rotations and split/join updates

```
splay(x):
    while x.parent!=null:
        p=x.parent; g=p.parent
        if g==null: rotate x over p
        else if (x is p.left)==(p is g.left):
            rotate p over g; rotate x over p
        else: rotate x over p; rotate x over g
    root=x
insert(k):
    if root==null: root=Node(k,parent=null); return
    search k; splay(found or last visited node)
    if root.key==k: update/reject duplicate; return
    if k<root.key:
        z=Node(k,left=root.left,right=root,parent=null); root.left=null
        if z.left!=null: z.left.parent=z
```


Complete splay rotations and split/join updates (continued)

```
    else:
        z=Node(k,left=root,right=root.right,parent=null); root.right=null
        if z.right!=null: z.right.parent=z
    root.parent=z; root=z
delete(k):
    search and splay target or last visited node; if absent: return
    (L,R)=(root.left,root.right)
    if L!=null: L.parent=null
    if R!=null: R.parent=null
    release root
    if L==null: root=R
    else:
        root=L; splay(maximum(L)); root.right=R
        if R!=null: R.parent=root
```

Empty insertion creates a leaf directly. Split/join restores both child and parent fields. Before releasing the old root, clear the parents of L and R so splay cannot follow a released ancestor. For chain 1→2→3→4, searching 4 first performs two left rotations over 2 and 3, then a zig over 1. The final tree is 4 with left child 1, 1.right=3, and 3.left=2, not a reversed chain.

Complete splay rotations and split/join updates (continued)

The access lemma proves searches. Updates additionally change weights and potential. With unit weights, split insertion adds a root rank at most log(n+1); severed subtrees only reduce existing ranks. Extra potential increase is O(log(n+1)). Deletion's split does not increase ranks, splaying max(L) uses the lemma, and attaching R increases only the new root rank by at most log n. Thus updates also have logarithmic amortized cost, without worst-case single-operation height or nonmutating-read guarantees.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Comparison and extensions

Structure	Objective	Search guarantee	Update characteristic
Optimal BST	Known static weighted cost	Optimal for the specified distribution	Rebuilding charged separately
AVL	Local height difference	O(log n) worst	Height maintenance, cascading deletion repairs
Red-black	Black height and red constraints	O(log n) worst	Color and rotation repair
Splay	Entire operation sequence	O(log n) amortized	Searches themselves rotate

Graduate extensions can use fixed nonuniform weights in the access lemma to study access-frequency costs, or investigate optimal BST root monotonicity. The first requires stating how weights relate to query counts. The complete Knuth-optimization proof is outside this edition.

Study checklist

- State each invariant, operation precondition and cost unit.
- AVL bounds worst-case height; splay bounds an operation sequence.
- For a rotation, check inorder order, parent links and required metadata.

References and attribution

Lin Cing, handwritten data structures notes. Sleator and Tarjan, Self-Adjusting Binary Search Trees, JACM 32(3), 1985, § 2: bottom-up splay and the access lemma. Numeric diagrams use this chapter's own keys and unit weights. The MIT 6.006 2020 index, L7, identifies AVL material. Princeton balanced search trees uses LLRB and its insertion/deletion code must not be mixed with general red-black conventions.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study